MODULE 4 — BACKEND ENGINEERING

Flask with Database Integration

Mastering relational persistence, SQLAlchemy ORM, CRUD pipelines, schema migrations, and transactional session control.

DATA PERSISTENCE

Why Do We Need a Database?

Without a Database

User Request → Flask (RAM Memory) → Data Lost on Restart

Variables in memory reset every time the server restarts or reloads.

With a Database

User Request → Flask App → Database Engine (Disk Storage)

Data remains safe across restarts, deployments, and hardware crashes.

Persistent Storage
Retains state indefinitely

Fast Querying
Filtering and aggregation

Security & Access
Encrypted user records

Data Integrity
Constraint validation

RDBMS FUNDAMENTALS

Relational Database Basics

SCHEMA EXAMPLE: STUDENT TABLE

2 Records • 4 Attributes

id (PK)	name	email	course
1	Rahul Sharma	rahul@gmail.com	MCA
2	Priya Patel	priya@gmail.com	BCA

Row / Record

One complete single item representing an entity instance.

Column / Field

Specific attribute type like name, email, or course string.

Primary Key (PK)

A unique identifier ensuring no two records collide.

Foreign Key (FK)

Relational link binding child tables to parent entities.

DATABASE ENGINE SELECTION

SQLite vs MySQL

Feature	SQLite	MySQL
Architecture	Serverless, stored directly in single disk file	Client-Server background database engine
Setup Cost	Zero config, built into Python standard library	Requires installation, port tuning & user grants
Concurrency	Database-level file lock on write transactions	Row-level locking for multi-client concurrency
Target Use Case	Prototyping, tutorials, local dev & testing	High-traffic web apps & production clusters

SQLite in Flask
Zero setup required; uses local file `app.db`

MySQL in Flask
Connects over network via `pymysql` driver

OBJECT RELATIONAL MAPPING

What is SQLAlchemy?

SQLAlchemy is a high-performance Python SQL toolkit and Object Relational Mapper that lets you operate on databases using pure Python objects instead of repetitive SQL strings.

Raw SQL Approach

```
cursor.execute(
  "SELECT * FROM student WHERE course = %s",
  ('MCA',)
)
rows = cursor.fetchall()
```

SQLAlchemy ORM Approach

```
students = Student.query.filter_by(
  course="MCA"
).all()
```

Pythonic Flow
No raw SQL strings

DB Abstraction
Switch SQLite / MySQL

Injection Safe
Parameterized input

Rapid CRUD
Instant model methods

CONCEPT ARCHITECTURE

ORM: How Does It Work?

Python Class ↔ Database Table

Object Instance ↔ Table Row (Record)

Class Attribute ↔ Table Column (Field)

PYTHON MODEL DEFINITION

```
class Student(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100))
    email = db.Column(db.String(100))
```

RESULTING SQL SCHEMA

```
CREATE TABLE student (
    id INTEGER NOT NULL PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
```

ENVIRONMENT CONFIGURATION

Flask + SQLAlchemy Setup

```
>_ pip install flask flask-sqlalchemy pymysql
```

```python
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

# Database URI connection configuration
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///students.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db = SQLAlchemy(app)
```

SQLite URI Format

```
sqlite:///students.db
```

Three slashes indicate relative path from application root.

MySQL URI Format

```
mysql+pymysql://user:pwd@localhost/dbname
```

Specifies dialect, driver, auth credentials, host, and database.

MODEL DECLARATION

Creating a Database Model

```python
class Student(db.Model):
    __tablename__ = 'students'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(100), unique=True, nullable=False)
    course = db.Column(db.String(50), default="MCA")

    def __repr__(self):
        return f"<Student {self.name}>"
```

`primary_key=True`
Unique index identifying each entity row.

`nullable=False`
Enforces NOT NULL constraint on database column.

`unique=True`
Prevents duplicate values across all rows.

`default=value`
Automatic fallback if value is omitted on insert.

SCHEMA INITIALIZATION

Creating Database Tables

TRIGGER TABLE CREATION VIA APPLICATION CONTEXT

```
with app.app_context():
    db.create_all()
```

Flask App
Loads app.config

→

app_context()
Binds active application

→

SQLAlchemy Engine
Inspects db.Model classes

→

Physical Schema
Tables created on disk

Note on Production Lifecycle

db.create_all() creates missing tables, but will NOT update existing columns when model attributes change. Use Flask-Migrate for real systems.

DATA MANIPULATION

CRUD Operations Overview

C

CREATE

Insert new entity into database

`INSERT INTO`

R

READ

Query records from database

`SELECT FROM`

U

UPDATE

Modify existing table record

`UPDATE SET`

D

DELETE

Purge record permanently

`DELETE FROM`

Web Request Cycle: HTTP Request → Flask Route View → SQLAlchemy ORM → Database Commit

CRUD STEP 1

CREATE: Insert Data

```
# 1. Instantiate the Python Model Object
new_student = Student(
    name="Rahul",
    email="rahul@gmail.com",
    course="MCA"
)

# 2. Stage object in current session
db.session.add(new_student)

# 3. Commit transaction to write disk
db.session.commit()
```

1. Instantiation

Creates regular Python class instance in local memory with provided field values.

2. Session Staging

`db.session.add()` marks model object as pending insertion inside transaction.

3. Transaction Commit

`db.session.commit()` converts staged operations into SQL INSERT and flushes to disk.

CRUD STEP 2

READ: Retrieve Data

Fetch All Records

```
students = Student.query.all()
```

Returns list of all model objects.

Fetch by Primary Key

```
student = db.session.get(Student, 1)
```

Returns single object or None.

Filter by Attribute

```
students = Student.query.filter_by(course="MCA").all()
```

Returns matches matching condition.

First Unique Match

```
student = Student.query.filter_by(email="r@g.com").first()
```

Stops scanning at first result.

Data Pipeline: `Database Table` → `SQLAlchemy Engine` → `Python Objects` → `Jinja2 / HTML Template`

CRUD STEP 3

UPDATE: Modify Data

```python
# 1. Retrieve the existing record
student = db.session.get(Student, 1)

# 2. Mutate attribute directly on Python object
if student:
    student.name = "Amit Verma"
    student.course = "Data Science"

    # 3. Commit triggers SQL UPDATE statement
    db.session.commit()
```

1. Find Record

Look up target record via primary key or unique attribute.

2. Change Attribute

SQLAlchemy automatically tracks dirty/modified attributes.

3. Commit Session

Database state synchronized with zero manual UPDATE strings.

Pro-Tip: No need to call `db.session.add()` when updating! The object is already attached to the active session.

CRUD STEP 4

DELETE: Remove Data

```python
# 1. Query the target record
student = db.session.get(Student, 1)

# 2. Stage object for deletion if it exists
if student:
    db.session.delete(student)

    # 3. Permanently execute deletion
    db.session.commit()
else:
    return "Student not found", 404
```

Permanent Deletion

Once committed, SQL DELETE is irrecoverable unless database backups exist.

Existence Validation

Always verify object is not None before invoking `db.session.delete()`.

Alternative: Soft Delete

Production apps often set a boolean flag `is_active = False` instead of hard deletion.

TRANSACTION SAFETY

Database Session Management

session.add()
Stages new object in transaction buffer.

session.commit()
Flushes pending operations to database disk.

session.rollback()
Cancels uncommitted changes after failure.

session.delete()
Marks entity for SQL DELETE removal.

PRODUCTION STANDARD: SAFE TRANSACTION PATTERN

```
try:
    db.session.add(new_student)
    db.session.commit()
    flash("Student created successfully!", "success")
except Exception as e:
    db.session.rollback()  # Restores clean database state
    flash("Database error occurred. Transaction aborted.", "danger")
```

SCHEMA EVOLUTION

Flask-Migrate

SETUP & EXTENSION BINDING

```
from flask_migrate import Migrate; migrate = Migrate(app, db)
```

```
pip install flask-migrate
```

`flask db init`

Initializes migrations directory structure once per project.

`flask db migrate -m "msg"`

Detects model changes and auto-generates migration script.

`flask db upgrade`

Applies pending migration scripts directly to database tables.

Workflow: 1. Edit Python Model → 2. flask db migrate → 3. Review Migration File → **4. flask db upgrade**

CAPSTONE OVERVIEW

Complete Flask Database Architecture & Hands-on Lab

END-TO-END REQUEST PIPELINE

1. Client Browser Form	HTTP POST
2. Flask Route & View Function	request.form
3. SQLAlchemy ORM Model	Class Instance
4. Session Management	add() & commit()
5. SQLite / MySQL Engine	**Disk Persistence**

HANDS-ON LAB: STUDENT PORTAL

Build a full Student Management Web App with the following capabilities:

- **Schema:** ID, Name, Email, Course with constraints
- **CRUD UI:** Add student, view list, search & delete
- **Migrations:** Initialize and upgrade via Flask-Migrate
- **Safety:** Graceful rollback on duplicate email error

Key Takeaway: Flask handles web routing, SQLAlchemy handles object mapping, and Flask-Migrate manages evolving database schemas.